Especificación de requisitos de software

La Unica

Centro de la Industria, la Empresa y los Servicios Regional Huila

Programa de formación Análisis y Desarrollo de Software

Ficha 3407799

Integrantes del grupo:

Camilo Andrés Tamayo Duran

Carlos Stiven Gutiérrez Ramírez

Kevin Fernando Martínez Pérez

Samuel Santiago Ramírez Fierro

Santiago Ortiz Claros

Instructor: José de Jesús Motta Vargas

30 de octubre de 2026

Control del documento

Control de la versión

Version	Fecha	Estado	Descripcion
1.0	05/10/2026	Reemplazada	Creacion del SRS y planteamiento de los requisitos funcionales (RF) y requisitos no funcionales (RNF) del sistema La Unica.
1.1	06/10/2026	Reemplazada	Corrección del modelo de datos: se eliminaron datos redundantes (copias del cliente y del producto, subtotal y total almacenados, id propio de detalle_factura) y se agregaron tipo_documento, unidad_medida, correo, direccion y cantidades decimales. Se ajustaron los requisitos funcionales, las reglas de negocio y los formatos.
1.2	26/10/2026	Reemplazada	Alineación de entidades, campos, estados, límites y reportes con la estructura de db.sql.
1.3	26/10/2026	Reemplazada	Actualización del stack tecnológico a PHP y MySQL.
1.4	27/10/2026	Reemplazada	Ajustes a reportes, reglas, dependencias y aceptación.

Version	Fecha	Estado	Descripcion
1.5	30/10/2026	Vigente	Alineación con el aplicativo: precio de venta guardado en detalle_facturas (migración 001), ventas por cobrar con registro de pago (Pendiente a Pagado), cancelación de facturas Pendiente, exclusión de facturas canceladas en los reportes de ventas, fecha de registro no futura y sección Sobre nosotros (RF-14 y RF-15). Se agregan la arquitectura del sistema (2.5), los requisitos de seguridad y de verificación automatizada (RNF-07 y RNF-08), la actualización de bases existentes (3.5.5) y se revisan tecnologías, restricciones, dependencias y criterios de aceptación.

Contenido

Control del documento....2
Resumen ejecutivo....7
1 Introducción....8
1.1 Planteamiento del problema....8
1.2 Propósito....8
1.3 Justificación del proyecto....8
1.4 Objetivo general....8
1.4.1 Objetivos específicos....9
1.5 Alcance del proyecto....9
1.6 Personal involucrado....11
1.7 Definiciones acrónimos y abreviaturas....12
1.7.1 Definiciones....12
1.7.2 Acrónimos....13
1.7.3 Abreviaturas....14
1.8 Referencias....14
1.9 Resumen....14
1.10 Fuentes primarias de información....15
2 Descripción general....16
2.1 Perspectiva del producto....16
2.2 Características de los usuarios....17
2.3 Restricciones....17
2.4 Suposiciones y dependencias....18

2.4.1 Suposiciones....18
2.4.2 Dependencias....19
2.5 Arquitectura del sistema....19
2.5.1 Capas del aplicativo....20
2.5.2 Estructura del repositorio....21
2.5.3 Flujo de una operación....22
2.5.4 Despliegue y configuración....22
2.5.5 Verificación automatizada....23
3 Requisitos específicos....24
3.1 Requisitos comunes de las interfaces....24
3.1.1 Interfaz de usuario....24
3.1.2 Interfaz de hardware....24
3.1.3 Interfaz de software....24
3.1.4 Interfaz de comunicación....25
3.2 Historias de usuario....25
3.2.1 Gestión de clientes....25
3.2.2 Gestión de productos....25
3.2.3 Generación de facturas....25
3.2.4 Consulta impresión y cancelación de facturas....26
3.2.5 Consulta de reportes....26
3.2.6 Cobro de ventas por cobrar....26
3.2.7 Consulta de ayuda....26
3.3 Requisitos funcionales....26

3.4 Requisitos no funcionales.....32

3.5 Modelo de datos y reglas de negocio.....36

3.5.1 Relaciones.....38

3.5.1.1 Vistas y procedimientos almacenados.....39

3.5.2 Reglas de negocio.....40

3.5.3 Formatos y límites de los datos.....42

3.5.4 Criterios de aceptación de la alineación con db.sql.....45

3.5.5 Actualización de bases existentes.....47

Referencias.....48

Resumen ejecutivo

La Unica será una aplicación web local para registrar clientes y productos, gestionar facturas y consultar reportes. La base de datos la_unica se crea con cuatro tablas (clientes, facturas, productos y detalle_facturas), tres vistas y cinco procedimientos almacenados definidos en db.sql.

La aplicación permitirá gestionar clientes identificados por id_cliente, productos con unidad de medida en texto, existencias enteras y precios vigentes, y facturas con detalles de cantidades enteras. Cada detalle guardará la cantidad y el precio_unitario aplicado en la venta: los subtotales y totales se calcularán con ese precio, de modo que actualizar el catálogo no altere las facturas emitidas. La aplicación en PHP implementará la lógica de venta, cobro, actualización de existencias y cancelación.

Una venta podrá registrarse de contado, con estado Pagado, o por cobrar, con estado Pendiente; ambas descontarán las existencias. Una factura Pendiente pasará a Pagado al registrar su pago, y una factura Pagado o Pendiente podrá cancelarse, lo que repondrá el inventario una sola vez. Los reportes de ventas excluirán las facturas canceladas. Este documento describe además la arquitectura del aplicativo, sus requisitos de seguridad y su verificación mediante pruebas automatizadas.

1 Introducción

1.1 Planteamiento del problema

Los registros manuales o dispersos de clientes, productos y ventas dificultan localizar información y controlar las existencias. Al elaborar una factura de forma manual pueden presentarse errores en las cantidades, los precios y el cálculo del total.

La Unica centralizará esta información y relacionará cada factura con su cliente y sus productos mediante facturas y detalle_facturas. Cada detalle registrará la cantidad y el precio_unitario aplicado al momento de la venta, para que el importe de una factura no cambie cuando se actualice el catálogo.

1.2 Propósito

Definir los requisitos de una aplicación web básica para registrar clientes y productos, generar y cobrar facturas, consultar reportes y utilizar las cuatro tablas, tres vistas y cinco procedimientos almacenados creados por db.sql, así como describir la arquitectura con la que se implementa.

1.3 Justificación del proyecto

El proyecto permitirá aplicar análisis de requisitos, diseño de bases de datos relacionales y desarrollo web mediante un proceso de venta de alcance controlado. El SRS refleja las columnas y restricciones que define db.sql y separa esas reglas de las validaciones y operaciones que implementará la aplicación en PHP.

1.4 Objetivo general

Diseñar y desarrollar La Unica como una aplicación web para gestionar clientes, productos y facturas, con detalle_facturas como relación entre las facturas y sus productos.

1.4.1 Objetivos específicos

- Registrar, consultar y actualizar clientes usando id_cliente, nombre y fecha_registro.
- Registrar, consultar y actualizar productos con su unidad de medida, precio vigente y existencias enteras.
- Generar facturas de contado o por cobrar asociadas a un cliente y a uno o varios productos.
- Registrar en detalle_facturas cantidades enteras y el precio_unitario vigente al momento de la venta, y calcular los importes a partir de ese precio guardado.
- Actualizar las existencias al completar o cancelar una factura.
- Consultar e imprimir facturas, registrar el pago de las ventas por cobrar y conservar el historial de ventas.
- Mantener la integridad de las relaciones entre las cuatro tablas y utilizar las vistas y procedimientos de reporte definidos en db.sql.
- Verificar automáticamente las reglas de venta, cobro, cancelación y reportes mediante pruebas de integración.

1.5 Alcance del proyecto

El aplicativo comprenderá los módulos de clientes, productos, facturación y reportes, además de una sección informativa (Sobre nosotros) con el manual de usuario, el video explicativo y la wiki del proyecto. Un operador podrá realizar las funciones desde el mismo aplicativo. Los clientes son compradores asociados a facturas, no cuentas de acceso.

Los clientes tendrán id_cliente y nombre; fecha_registro se asignará con la fecha actual si no se proporciona y, si se proporciona, no podrá ser posterior a la fecha actual. Los productos tendrán nombre, unidad_medida como texto de hasta 20 caracteres, precio_unitario no negativo y stock entero no negativo. El esquema no define documentos, teléfonos, correos, direcciones, códigos ni descripciones.

Para generar una factura, el operador seleccionará un cliente existente, elegirá la forma de pago (de contado o por cobrar), agregará productos e indicará cantidades enteras positivas. La base asignará id_factura, fecha_factura tendrá por defecto la fecha y hora del servidor, y cada detalle tendrá un id_detalle_factura propio. Al completar una venta, la aplicación en PHP guardará explícitamente el estado Pagado o Pendiente, copiará al detalle el precio_unitario vigente de cada producto y actualizará las existencias. El total se calculará con los precios guardados en detalle_facturas; no se guardará un número FAC ni un total almacenado.

La Unica es un prototipo formativo. En este alcance, la factura será un comprobante interno de venta calculado como cantidad por el precio_unitario registrado en cada detalle; no incluirá cálculo de IVA u otros impuestos y no se presentará como factura fiscal o electrónica. Esta simplificación delimita el ejercicio académico y no incorpora reglas tributarias al aplicativo.

El alcance no incluirá administración de usuarios o roles, autenticación por cuentas, recuperación de contraseñas, gestión de categorías, tabla de tienda, contabilidad avanzada, descuentos, pasarelas de pago, facturación electrónica ante servicios externos ni aplicaciones móviles nativas. Las operaciones persistentes utilizarán las cuatro tablas; los reportes podrán consultar las vistas y procedimientos

incluidos en db.sql. Registrar el pago de una factura Pendiente solo cambiará su estado; no procesará medios de pago.

1.6 Personal involucrado

El desarrollo estará a cargo de los cinco integrantes del grupo. Cada integrante tendrá una responsabilidad principal y podrá apoyar otras actividades del proyecto cuando sea necesario.

Integrantes, roles y responsabilidades del proyecto

Nombre	Rol principal	Responsabilidades
Carlos Stiven Gutiérrez Ramírez	Desarrollador Frontend y UX/UI	Diseñar y desarrollar las interfaces, navegación, componentes visuales y adaptación responsive.
Camilo Andrés Tamayo Duran	Desarrollador Backend e Integraciones	Desarrollar la lógica del servidor, servicios y procesos de negocio de la aplicación.
Kevin Fernando Martínez Pérez	Analista de Requisitos y Documentación	Mantener el SRS, organizar requisitos, trazabilidad y documentación técnica.
Samuel Santiago Ramírez Fierro	Responsable de Base de Datos	Diseñar y mantener la estructura de datos, relaciones, consultas e integridad de la información.
Santiago Ortiz Claros	Responsable de Calidad, Pruebas y Seguridad	Diseñar y ejecutar pruebas, registrar errores, validar requisitos y apoyar la revisión de seguridad.

1.7 Definiciones acrónimos y abreviaturas

1.7.1 Definiciones

Termino	Definicion
Cliente	Comprador registrado en clientes; puede no tener facturas o estar asociado a una o varias.
Operador	Persona de la tienda que registra clientes y productos y realiza las operaciones de facturación. No constituye una tabla del modelo.
Producto	Artículo registrado en productos con nombre, unidad_medida, precio_unitario y stock. La unidad se almacena como texto; el esquema no fija una lista de valores.
Factura	Registro de facturas asociado a un cliente, con id_factura, fecha_factura y estado_factura. El total no se almacena: se calcula sumando cantidad por el precio_unitario guardado en cada detalle.
Detalle de factura	Fila de detalle_facturas que enlaza una factura con un producto mediante id_detalle_factura, id_factura e id_producto, y almacena la cantidad entera y el precio_unitario aplicado en la venta. No guarda subtotal.
Stock	Existencia entera no negativa del producto, limitada por el tipo INT y la restricción CHECK de db.sql.
Unidad de medida	Texto almacenado en unidad_medida (VARCHAR(20)); db.sql no define una lista cerrada ni habilita cantidades decimales.

Termino	Definicion
Tabla pivote	detalle_facturas relaciona facturas y productos. Cada fila tiene id_detalle_factura como clave primaria; no se declara unicidad para el par id_factura e id_producto.
Vista	Consulta guardada creada por db.sql para resumir información de clientes, facturas o productos.
Procedimiento almacenado	Rutina de consulta creada por db.sql para generar reportes.
Venta por cobrar	Factura registrada con estado Pendiente: descuenta existencias y queda pendiente de pago hasta que el operador registre el pago (Pagado) o la cancele.
Comprobante interno	Vista imprimible de una factura con cliente, fecha, estado, detalles, precios registrados y total. No es una factura fiscal ni electrónica.
Transacción	Conjunto de operaciones de base de datos que se confirma completo o se revierte completo, sin dejar cambios parciales.
Migración	Script SQL que actualiza una base creada con una versión anterior de db.sql sin perder sus datos; se ejecuta una sola vez.
Prueba de integración	Prueba automatizada que ejecuta funciones del aplicativo contra una base de datos de prueba y verifica el resultado.

1.7.2 Acrónimos

- SRS: Software Requirements Specification.
- RF: Requisito funcional.

- RNF: Requisito no funcional.
- BD: Base de datos.
- SQL: Structured Query Language.
- HTTP: Hypertext Transfer Protocol.
- CSRF: Cross-Site Request Forgery (falsificación de solicitudes entre sitios).
- XSS: Cross-Site Scripting (inyección de código en páginas web).
- PRG: Post/Redirect/Get (patrón de redirección tras un envío de formulario).
- CI: Continuous Integration (integración continua).
- COP: peso colombiano.

1.7.3 Abreviaturas

- PK: Clave primaria.
- FK: Clave foránea.
- 1-N: Relación uno a muchos.
- N-N: Relación muchos a muchos.

1.8 Referencias

La organización documental y los datos del grupo e instructor proceden del documento base del proyecto (Especificación de requisitos de software [SRS]: La Unica, 2026). La versión 1.5 se contrastó con el código del aplicativo, el script db.sql, la migración 001_precio_historico.sql y las pruebas automatizadas del repositorio (Equipo del proyecto, 2026).

1.9 Resumen

La Unica permitirá gestionar clientes y productos y registrar ventas de contado o por cobrar mediante facturas y detalle_facturas. Este documento describe las cuatro

tablas, las tres vistas y los cinco procedimientos almacenados del esquema, la arquitectura del aplicativo y las reglas de aplicación necesarias para operar la venta, el cobro y la cancelación.

1.10 Fuentes primarias de información

Los requisitos se fundamentan en la definición del proyecto, en la estructura de db.sql y en el código del aplicativo. El archivo crea la base la_unica, cuatro tablas con sus claves y restricciones, tres vistas y cinco procedimientos almacenados; no inserta datos iniciales. Las bases creadas con una versión anterior se actualizan con referencias/migraciones/001_precio_historico.sql.

2 Descripción general

2.1 Perspectiva del producto

La Unica será una aplicación web local para una tienda y se ejecutará en el equipo del operador. La aplicación se desarrollará con PHP y MySQL: PHP implementará la lógica y MySQL almacenará la base la_unica, con las tablas, vistas y procedimientos definidos en db.sql. El navegador accederá al servicio por la interfaz de bucle local del mismo equipo. La arquitectura se describe en la sección 2.5.

Tecnologías seleccionadas

Elemento	Tecnologia
Lenguaje de programación	PHP
Motor de base de datos	MySQL (MariaDB en las pruebas automatizadas)
Servidor web	Apache (entorno local Laragon)
Interfaz	HTML, CSS propio y JavaScript sin bibliotecas externas; fuentes locales
Acceso a datos	Extensión mysqli con sentencias preparadas
Pruebas automatizadas	PHPUnit 11, instalado con Composer
Control de versiones e integración continua	Git, GitHub y GitHub Actions

2.2 Características de los usuarios

Participantes del proceso y su relacion con el aplicativo

Participante	Caracteristicas	Funciones o relacion con el aplicativo
Operador de la tienda	Persona con conocimientos básicos de manejo de formularios y ventas.	Registrar, consultar y actualizar clientes y productos; generar ventas de contado o por cobrar; consultar, imprimir, cobrar y cancelar facturas; consultar reportes y la ayuda del aplicativo.
Cliente	Persona que realiza una compra y cuyos datos se registran en clientes.	Quedar asociado a sus facturas y recibir la factura impresa. No opera el aplicativo ni tiene una cuenta.

2.3 Restricciones

- La aplicación será web y utilizará PHP 7.4 o posterior, con mysqli y mysqlnd, y MySQL 8.0.16 o posterior (o MariaDB 10.11 o posterior) para aplicar las restricciones CHECK del esquema. Las pruebas automatizadas requieren PHP 8.2 o posterior y Composer.
- El esquema de persistencia utilizará las cuatro tablas clientes, facturas, productos y detalle_facturas, además de las vistas y procedimientos que crea db.sql.
- La relación N-N entre facturas y productos se resolverá mediante detalle_facturas, que tiene una clave primaria propia y no declara unicidad para el par de claves foráneas.

- Cada factura se asociará a un cliente existente por id_cliente. La regla de que contenga al menos un detalle corresponde a la aplicación en PHP; db.sql no la impone mediante una restricción.
- No se implementarán módulos que requieran tablas adicionales.
- Los reportes y las impresiones calcularán importes con el precio_unitario guardado en detalle_facturas al registrar la venta. Cambiar el precio de un producto solo afectará las ventas posteriores.
- El aplicativo no usará frameworks ni bibliotecas externas; las dependencias de Composer se limitan a las pruebas automatizadas.
- Como el aplicativo no autentica al operador, Apache y MySQL deberán escuchar solo en la interfaz de bucle local (RNF-03).

2.4 Suposiciones y dependencias

2.4.1 Suposiciones

- El aplicativo y la base de datos se ejecutarán en el equipo del operador, sin acceso desde otros equipos ni publicación en Internet. El acceso al equipo se administrará mediante el sistema operativo; el aplicativo no autenticará al operador.
- El operador registrará datos válidos y mantendrá actualizados los clientes, productos y existencias. La aplicación validará formatos y reglas que db.sql no impone.
- Los clientes se registrarán antes de completar una factura.
- Los equipos dispondrán de un navegador compatible; la impresión utilizará las opciones del navegador.

- Si el puerto, el usuario o la contraseña de MySQL difieren de los valores predeterminados (127.0.0.1, puerto 3306, usuario root sin contraseña), se configurarán en php/config/conexion.local.php o mediante las variables de entorno DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME y DB_PASSWORD.
- db.sql no fija el juego de caracteres, la intercalación, el motor de almacenamiento ni la zona horaria de MySQL. El despliegue configurará estos valores; para conservar texto en español se utilizará utf8mb4, y para presentar fechas de Bogotá la aplicación en PHP configurará explícitamente la zona horaria de la aplicación y de la sesión de base de datos.

2.4.2 Dependencias

- Servidor local Apache capaz de ejecutar PHP 7.4 o posterior con las extensiones mysqli y mysqlnd habilitadas, y con AllowOverride activo para aplicar el archivo .htaccess del proyecto.
- Servicio MySQL 8.0.16 o posterior (o MariaDB 10.11 o posterior), con soporte para las restricciones CHECK y DEFAULT (CURRENT_DATE) de db.sql.
- Para el desarrollo: Composer con PHPUnit 11 (PHP 8.2 o posterior), una base de datos de prueba cuyo nombre empiece por la_unica_test y un repositorio en GitHub con GitHub Actions para la integración continua.

2.5 Arquitectura del sistema

La Unica seguirá una arquitectura cliente-servidor local de tipo monolítico: un único aplicativo PHP, servido por Apache en el equipo del operador, atenderá las

solicitudes del navegador y accederá a MySQL. No habrá una API externa ni servicios remotos; el navegador y el servidor se comunicarán por la interfaz de bucle local.

2.5.1 Capas del aplicativo

El código se organizará en tres capas lógicas. Cada página PHP actuará como controlador de su ruta: validará la solicitud, invocará la lógica correspondiente y presentará la vista.

Capas del aplicativo

Capa	Componentes	Responsabilidad
Presentación	php/includes/menu.php y pie.php, vistas PHP de cada módulo, assets/css/estilos.css y assets/js/facturas.js.	Generar HTML escapado, formularios con token CSRF, el menú adaptable y el total estimado de la venta en el navegador.
Lógica de aplicación	Páginas de cada módulo, php/facturas/servicio_facturas.php y php/config/conexion.php.	Validar entradas, aplicar las reglas de negocio, ejecutar ventas, cobros y cancelaciones en transacciones y redirigir con el patrón PRG.
Datos	Tablas, vistas y procedimientos de db.sql; php/config/consultas.php y php/reportes/datos_reportes.php.	Persistir la información, imponer claves y restricciones CHECK y entregar los datos de los reportes.

2.5.2 Estructura del repositorio

La siguiente tabla resume los directorios del repositorio y la función de cada uno.

Directorios del repositorio

Ruta	Contenido
index.php y main.php	Punto de entrada y panel de inicio.
php/config/	entorno.php (URL base, cabeceras de seguridad y escape HTML), conexion.php (conexión, sesión y validaciones), consultas.php (consultas preparadas) y conexion.local.example.php (plantilla de configuración local).
php/clientes/ y php/productos/	Registro, consulta, edición y eliminación protegida.
php/facturas/	Venta, consulta, comprobante, registro de pago, cancelación y servicio_facturas.php con las reglas de negocio.
php/reportes/	Resumen, plantilla común y siete reportes definidos en datos_reportes.php.
php/sobre_nosotros/	Manual de usuario en PDF y video explicativo; no requieren la base de datos.
assets/	Hoja de estilos, fuentes locales, logotipo, JavaScript y video.
wiki/	Documentación de uso, configuración y notas técnicas.

Ruta	Contenido
referencias/	db.sql, migraciones, usuario_app.sql, SRS, manual de usuario y documentación de QA.
tests/	Pruebas de integración con PHPUnit.
.github/workflows/	Validación de sintaxis y pruebas en GitHub Actions.

2.5.3 Flujo de una operación

Toda operación que modifique datos seguirá el mismo flujo. El navegador enviará el formulario por POST con el token CSRF de la sesión. La página validará el token, los tipos y los límites. La lógica se ejecutará con sentencias preparadas y, si afecta existencias o estados, dentro de una transacción que bloquea las filas implicadas (SELECT … FOR UPDATE), con los productos en orden ascendente de id_producto para evitar interbloqueos.

Si la operación se completa, la transacción se confirmará. Al crear un registro, registrar un pago o cancelar una factura, el servidor responderá con una redirección HTTP 303 (patrón PRG) y mostrará el resultado una sola vez mediante un mensaje guardado en la sesión; la edición y la eliminación mostrarán el resultado en la misma página. Si una regla no se cumple, la transacción se revertirá y el mensaje aparecerá junto al formulario, conservando los datos ingresados. Los errores técnicos se registrarán en el log del servidor y no se mostrarán al operador.

2.5.4 Despliegue y configuración

El aplicativo se instalará en la carpeta servida por Apache, por ejemplo la de Laragon. La conexión usará por defecto 127.0.0.1, puerto 3306, base la_unica y usuario root sin contraseña; cada equipo podrá cambiar estos valores en

php/config/conexion.local.php, que no se incluye en el control de versiones, o con variables de entorno DB_*. Se recomienda el usuario la_unica_app, creado con referencias/usuario_app.sql, con permisos limitados a la base la_unica. El archivo .htaccess impedirá servir el historial de Git, la configuración, las pruebas, los scripts SQL y los documentos internos.

2.5.5 Verificación automatizada

Las reglas de venta, cobro, cancelación, reportes y restricciones de la base se verificarán con pruebas de integración en PHPUnit, ejecutadas contra una base de prueba cuyo nombre empieza por la_unica_test. En cada envío al repositorio, GitHub Actions ejecutará la validación de sintaxis de PHP y JavaScript, las pruebas sobre una base creada con db.sql y las mismas pruebas sobre una base migrada desde el esquema anterior.

3 Requisitos específicos

3.1 Requisitos comunes de las interfaces

3.1.1 Interfaz de usuario

La interfaz ofrecerá acceso a Clientes, Productos, Facturas, Reportes y Sobre nosotros desde un menú lateral que se adapta a pantallas pequeñas. La sección Reportes mostrará un resumen con los siete indicadores definidos en RF-13 y acceso a cada reporte individual. Los formularios de clientes solicitarán id_cliente, nombre y, de forma opcional, fecha_registro; los de productos, nombre, unidad_medida, precio_unitario y stock. La preparación de una factura mostrará cliente, forma de pago (de contado o por cobrar), productos, cantidades y un total estimado con el precio vigente. Los productos con stock cero podrán mostrarse como agotados. El comprobante de una factura Pendiente ofrecerá Registrar pago, y el de una factura Pagado o Pendiente, Cancelar factura. La cancelación estará deshabilitada para facturas Cancelado; la eliminación estará deshabilitada cuando haya referencias asociadas.

3.1.2 Interfaz de hardware

El sistema podrá utilizarse desde un equipo con navegador web. No requerirá hardware especializado. Para obtener una copia física de la factura se utilizará una impresora disponible en el equipo del operador.

3.1.3 Interfaz de software

La aplicación desarrollada en PHP se comunicará con MySQL para leer y guardar clientes, facturas, productos y detalle_facturas, así como para consultar las vistas y ejecutar los procedimientos de reporte definidos en db.sql. El servicio se

ejecutará localmente en el equipo del operador. Los parámetros de conexión tendrán valores predeterminados en php/config/conexion.php, que podrán sustituirse con php/config/conexion.local.php o con variables de entorno DB_*.

3.1.4 Interfaz de comunicación

La comunicación entre navegador y aplicación se realizará mediante solicitudes HTTP en el mismo equipo: GET para las consultas y POST, con token CSRF, para las operaciones que modifican datos. Tras crear un registro, registrar un pago o cancelar una factura, el servidor responderá con una redirección 303, de modo que recargar la página no repita la operación. El servidor validará los datos antes de guardarlos y devolverá mensajes claros si una operación no puede completarse.

3.2 Historias de usuario

3.2.1 Gestión de clientes

Como operador, quiero registrar, buscar y actualizar clientes por id_cliente y nombre para seleccionar al comprador de una venta y conservar su relación con las facturas.

3.2.2 Gestión de productos

Como operador, quiero registrar y actualizar productos con nombre, unidad_medida, precio_unitario y stock entero para preparar ventas con datos vigentes, sabiendo que un cambio de precio no altera las facturas ya emitidas. Quiero ver los productos agotados y eliminar solo productos que no tengan detalles asociados.

3.2.3 Generación de facturas

Como operador, quiero seleccionar un cliente registrado, indicar si la venta es de contado o por cobrar y agregar productos con cantidades enteras positivas para

generar una factura que guarde el precio de cada producto y cuyas existencias se actualicen al completar la venta.

3.2.4 Consulta impresión y cancelación de facturas

Como operador, quiero consultar e imprimir facturas por su id_factura, cliente o fecha. Los importes utilizarán el precio registrado en la venta. Quiero cancelar facturas Pagado o Pendiente y que la acción quede bloqueada si ya están Cancelado.

3.2.5 Consulta de reportes

Como operador, quiero consultar reportes de clientes, ventas, existencias y estados de factura para revisar los datos consolidados por el esquema.

3.2.6 Cobro de ventas por cobrar

Como operador, quiero consultar las facturas pendientes y registrar el pago de una de ellas cuando el cliente pague, para que pase a Pagado sin modificar las existencias.

3.2.7 Consulta de ayuda

Como operador, quiero consultar desde el aplicativo el manual de usuario, el video explicativo y la wiki para resolver dudas sin salir del equipo, incluso si la base de datos no está disponible.

3.3 Requisitos funcionales

Los requisitos funcionales describen las operaciones que el aplicativo ofrecerá al operador. Cada identificador corresponde al alcance de esta versión.

Requisitos funcionales del sistema

ID	Requisito funcional	Descripcion
RF-01	Registrar clientes	Guardar id_cliente (INT suministrado por la aplicación) y nombre (VARCHAR(100) obligatorio). Si no se envía fecha_registro, MySQL asignará CURRENT_DATE; si se envía, deberá ser una fecha válida no posterior a la fecha actual. El esquema no define documento, teléfono, correo, dirección ni unicidad del nombre.
RF-02	Consultar y actualizar clientes	Buscar por id_cliente o nombre y permitir actualizar el nombre. Las facturas relacionarán al cliente por id_cliente; al consultar, se mostrará el nombre vigente. La clave foránea impide borrar clientes referenciados.
RF-03	Registrar productos	Guardar nombre (VARCHAR(100)), unidad_medida (VARCHAR(20)), precio_unitario (DECIMAL(10,2)) y stock (INT). Exigir precio y stock no negativos; el esquema no define códigos, descripciones ni una lista cerrada de unidades.

ID	Requisito funcional	Descripcion
RF-04	Consultar y actualizar productos	Buscar por id_producto o nombre y actualizar nombre, unidad_medida, precio_unitario o stock con los tipos y límites de db.sql. La unidad no podrá cambiarse si el producto aparece en detalles. Un cambio de precio se aplicará solo a ventas posteriores. Las claves foráneas impiden borrar productos referenciados.
RF-05	Preparar una factura	Seleccionar un cliente, la forma de pago y productos existentes, e indicar cantidades enteras positivas, entre 1 y 100 líneas. Mostrar el precio_unitario vigente y un total estimado, y validar el stock disponible antes de guardar. La unidad_medida es texto y no modifica el tipo entero de stock o cantidad.
RF-06	Generar la factura	Guardar una fila en facturas con id_factura autogenerado, id_cliente existente, fecha_factura por defecto CURRENT_TIMESTAMP y estado explícito: Pagado para una venta de contado o Pendiente para una venta por cobrar. Guardar una o más filas en detalle_facturas con cantidad entera positiva y el precio_unitario vigente del producto en ese momento. Si el estado se omite directamente en MySQL, se aplica Pendiente.

ID	Requisito funcional	Descripcion
RF-07	Calcular importes	Calcular subtotal y total al consultar, multiplicando la cantidad por el precio_unitario guardado en detalle_facturas. Una actualización posterior del precio del producto no modifica los importes de las facturas existentes. El esquema no almacena subtotal ni total.
RF-08	Controlar existencias	Mostrar el stock entero vigente, validar que alcance para la venta y actualizarlo en la aplicación en PHP al guardar. La restricción CHECK de db.sql impide stock negativo en motores que apliquen CHECK, pero db.sql no implementa el descuento ni una transacción de venta.
RF-09	Consultar e imprimir facturas	Buscar por id_factura, id_cliente o nombre del cliente, y fecha_factura. Mostrar el estado, la fecha, los detalles, el precio_unitario registrado en la venta, los subtotales y el total calculado. No se generará un número FAC.
RF-10	Cancelar facturas	Permitir en la aplicación en PHP las transiciones Pagado a Cancelado y Pendiente a Cancelado, con confirmación, y reponer las cantidades una sola vez. Bloquear la cancelación repetida.

ID	Requisito funcional	Descripcion
RF-11	Validar los registros	Validar los campos obligatorios, tipos y límites definidos: nombre de hasta 100 caracteres, unidad de medida de hasta 20, precio DECIMAL(10,2) no negativo, stock entero no negativo, cantidad entera positiva y fecha_registro no posterior a la fecha actual. El esquema no declara unicidad para nombres ni para el par factura-producto.
RF-12	Eliminar registros sin historial	Solicitar confirmación y eliminar clientes o productos solo cuando no tengan referencias en las claves foráneas. No borrar facturas ni detalles desde la aplicación. db.sql no define reglas ON DELETE CASCADE.

ID	Requisito funcional	Descripcion
RF-13	Consultar reportes	Incluir una portada con resumen y enlaces a siete reportes individuales: «Clientes y cantidad de facturas», «Total por factura», «Unidades vendidas por producto», «Detalle de facturas», «Existencias de productos», «Facturas por estado» y «Facturas pendientes». El resumen mostrará la cantidad de clientes y de sus facturas (todos los estados); el importe de las facturas Pagado y Pendiente; las unidades vendidas en esas facturas; las líneas de detalle y facturas distintas (todos los estados); las existencias totales; el total y desglose por estado; y la cantidad de facturas pendientes. Mantener los cálculos y filtros de 3.5.1.1.
RF-14	Registrar el pago de facturas	Permitir en la aplicación en PHP la transición Pendiente a Pagado desde el comprobante, con confirmación y token CSRF, sin modificar las existencias. Rechazar la operación si la factura no está Pendiente.

ID	Requisito funcional	Descripcion
RF-15	Consultar la ayuda del aplicativo	Ofrecer en Sobre nosotros el manual de usuario en PDF integrado, la página del video explicativo (assets/video/1.mp4) y el acceso a la wiki del proyecto. Estas páginas no requerirán conexión a la base de datos.

3.4 Requisitos no funcionales

Los requisitos no funcionales establecen las condiciones de calidad y funcionamiento del aplicativo.

Requisitos no funcionales del sistema

ID	Categoria	Especificacion
RNF-01	Usabilidad	Las acciones principales estarán en Clientes, Productos, Facturas y Reportes. Los errores identificarán el campo y la causa. Los productos agotados y las acciones bloqueadas por historial o estado serán visibles.

ID	Categoria	Especificacion
RNF-02	Rendimiento	En el equipo local y sin operaciones concurrentes, se medirán al menos 100 ejecuciones completas de cada consulta por id_cliente, id_producto e id_factura, desde el envío hasta mostrar los resultados. La base de prueba tendrá 1.000 clientes, 1.000 productos y 1.000 facturas, con un máximo de diez detalles por factura. Cumple si el percentil 95 del tiempo de respuesta de cada consulta es de dos segundos o menos.
RNF-03	Exposición del servicio	La aplicación y MySQL escucharán únicamente en la interfaz de bucle local: Apache se configurará con Listen 127.0.0.1 y MySQL con bind-address 127.0.0.1. No se expondrán rutas de datos ni el puerto de MySQL a la red local o a Internet. El control de acceso al equipo corresponde al sistema operativo.

ID	Categoria	Especificacion
RNF-04	Integridad	La aplicación en PHP guardará ventas, pagos y cancelaciones con sus cambios de stock en transacciones y validaciones para evitar escrituras parciales; las ventas bloquearán los productos en orden ascendente de id_producto. Los ajustes manuales se aplicarán en una operación atómica por producto y se coordinarán con ventas y cancelaciones concurrentes para evitar actualizaciones perdidas o stock negativo. db.sql define claves foráneas y CHECK de stock/cantidad, pero no contiene procedimientos de venta, bloqueo ni reposición de existencias.
RNF-05	Compatibilidad	La interfaz funcionará en navegadores web modernos, adaptará el menú a pantallas pequeñas y permitirá imprimir el comprobante interno con las funciones habituales del navegador, conservando sus datos e importes.

ID	Categoria	Especificacion
RNF-06	Mantenibilidad	El código se organizará en PHP sin frameworks, con la lógica de facturación separada de las vistas en servicio_facturas.php, y la estructura de la base de datos se reproducirá con db.sql: cuatro tablas, tres vistas y cinco procedimientos almacenados. Los cambios de esquema para bases existentes se entregarán como migraciones en referencias/migraciones. db.sql no inserta datos y no define charset, collation, motor, índices únicos de negocio ni triggers.
RNF-07	Seguridad	Las consultas con datos del operador usarán sentencias preparadas y toda salida HTML se escapará. Las operaciones que modifican datos exigirán POST y token CSRF. La cookie de sesión será HttpOnly y SameSite=Strict, y las páginas enviarán X-Frame-Options, X-Content-Type-Options y Referrer-Policy. Los errores técnicos se registrarán en el log sin mostrarse al operador. El aplicativo podrá usar un usuario de MySQL con permisos limitados a la_unica, y Apache no servirá el historial de Git, la configuración, las pruebas ni los scripts SQL.

ID	Categoria	Especificacion
RNF-08	Verificabilidad	Las reglas de venta, cobro, cancelación, reportes y restricciones de la base contarán con pruebas de integración automatizadas (PHPUnit) sobre una base de prueba independiente. Cada envío al repositorio ejecutará en GitHub Actions la validación de sintaxis de PHP y JavaScript y las pruebas, incluida una base migrada desde el esquema anterior. Cumple si todas las pruebas terminan sin fallos.

3.5 Modelo de datos y reglas de negocio

El modelo relacional coincide con db.sql: cuatro tablas con las columnas y tipos que aparecen a continuación. El archivo también crea tres vistas y cinco procedimientos almacenados. Las reglas sobre transacciones, estados de la venta y cambios de stock que no estén expresadas como restricciones en SQL corresponden a la aplicación en PHP.

Tabla	Campos	Finalidad
clientes	id_cliente INT (PK, sin autoincremento), nombre VARCHAR(100) NOT NULL, fecha_registro DATE DEFAULT CURRENT_DATE.	Registrar clientes y su fecha de registro.

Tabla	Campos	Finalidad
productos	id_producto INT AUTO_INCREMENT (PK), nombre VARCHAR(100) NOT NULL, unidad_medida VARCHAR(20) NOT NULL, stock INT NOT NULL CHECK (stock >= 0), precio_unitario DECIMAL(10,2) NOT NULL CHECK (precio_unitario >= 0).	Registrar productos, existencias enteras y el precio vigente para nuevas ventas.
facturas	id_factura INT AUTO_INCREMENT (PK), id_cliente INT NOT NULL (FK), estado_factura ENUM(Pagado, Cancelado, Pendiente) DEFAULT Pendiente, fecha_factura DATETIME DEFAULT CURRENT_TIMESTAMP.	Registrar una factura asociada a un cliente, con estado y fecha. No hay campo de número ni total.

Tabla	Campos	Finalidad
detalle_facturas	id_detalle_factura INT AUTO_INCREMENT (PK), id_factura INT NOT NULL (FK), id_producto INT NOT NULL (FK), cantidad INT NOT NULL CHECK (cantidad > 0), precio_unitario DECIMAL(10,2) NOT NULL CHECK (precio_unitario >= 0).	Relacionar factura y producto, con la cantidad y el precio aplicado en la venta. No almacena subtotal ni total.

3.5.1 Relaciones

- clientes 1-N facturas: un cliente puede tener varias facturas y cada factura referencia un id_cliente existente.
- facturas 1-N detalle_facturas: cada fila de detalle_facturas referencia una factura existente.
- productos 1-N detalle_facturas: un producto puede aparecer en varias filas de detalle_facturas.
- facturas N-N productos: detalle_facturas relaciona ambas tablas y utiliza id_detalle_factura como clave primaria. db.sql no declara unicidad para (id_factura, id_producto), por lo que permite más de una fila del mismo producto en una factura.

3.5.1.1 Vistas y procedimientos almacenados

Objeto	Datos y comportamiento definidos en db.sql
vista_clientes_total_facturas	Cuenta todas las facturas de cada cliente; incluye clientes sin facturas y no filtra por estado. El aplicativo usa el procedimiento equivalente; la vista se conserva para consultas directas y pruebas.
vista_total_facturas	Calcula el total por factura sumando cantidad por el precio_unitario guardado en detalle_facturas. Usa uniones internas, incluye el estado y excluye las facturas Cancelado.
vista_unidades_productos	Suma las cantidades de detalle por producto, excluyendo las facturas Cancelado.
reporte_detalle_facturas()	Devuelve factura, cliente, estado, fecha, producto, cantidad, precio registrado y subtotal de todos los estados; ordena por id_factura y por línea.
reporte_facturas_clientes()	Cuenta todas las facturas por cliente, incluye clientes sin facturas y ordena de mayor a menor total.
reporte_stock_productos()	Devuelve id, nombre, stock y precio vigente; ordena por stock ascendente.
reporte_facturas_por_estado()	Cuenta facturas agrupadas por estado, incluidos los estados presentes en la tabla.
reporte_facturas_pendientes()	Lista facturas con estado Pendiente, junto con cliente y fecha; ordena por fecha ascendente.

3.5.2 Reglas de negocio

- Cada fila de facturas debe referenciar un cliente mediante id_cliente y cada fila de detalle_facturas debe referenciar una factura y un producto existentes. Las claves foráneas de db.sql protegen estas relaciones. La exigencia de al menos un detalle por venta se validará en la aplicación en PHP, ya que el esquema no la impone.
- unidad_medida se almacena como VARCHAR(20); db.sql no enumera valores permitidos ni relaciona la unidad con cantidades decimales. stock y cantidad son INT: stock no puede ser negativo y cantidad debe ser mayor que cero según los CHECK definidos. La aplicación validará estas reglas antes de guardar.
- Al guardar, la aplicación en PHP volverá a validar el cliente y el stock disponible bajo bloqueo, y copiará al detalle el precio_unitario vigente de cada producto. db.sql no contiene lógica para reservar o descontar existencias; esas operaciones se implementarán en la aplicación.
- facturas guarda id_cliente y detalle_facturas guarda id_producto, cantidad y precio_unitario. El esquema no copia el nombre del cliente ni el del producto; las consultas muestran los nombres vigentes en las tablas relacionadas.
- Actualizar productos.precio_unitario solo afecta las ventas posteriores: los subtotales y totales de las facturas existentes se calculan con el precio guardado en su detalle. Las consultas de cliente muestran el nombre vigente del registro asociado.

- El ajuste manual fijará stock a un entero no negativo en una actualización atómica con bloqueo de la fila. Si el operador no modifica el stock al editar un producto, se conservará el valor vigente; si lo modifica y otra operación cambió las existencias mientras editaba, la aplicación rechazará el ajuste y mostrará el valor actual. db.sql no implementa esa coordinación.
- Al cancelar, la aplicación en PHP sumará al stock vigente las cantidades enteras del detalle, dentro de una operación atómica. Las transiciones de estado y la reposición única son reglas de aplicación, no restricciones del esquema.
- facturas.estado_factura admite Pagado, Cancelado y Pendiente, y el valor predeterminado del esquema es Pendiente. La aplicación en PHP escribirá Pagado en una venta de contado y Pendiente en una venta por cobrar, y solo permitirá las transiciones Pendiente a Pagado (registro de pago), Pagado a Cancelado y Pendiente a Cancelado. Cancelado es un estado final.
- Los reportes calculan el subtotal como cantidad por el precio_unitario guardado en el detalle y el total como suma de subtotales. Ninguno se almacena. Los reportes de ventas (total por factura y unidades vendidas) excluyen las facturas Cancelado, cuyas unidades ya volvieron al inventario. La aplicación usará valores decimales con dos posiciones para presentar los importes.
- La aplicación en PHP guardará factura, detalles y actualización de stock dentro de una transacción para evitar cambios parciales. db.sql define restricciones de claves y CHECK, pero no implementa esa transacción.

- detalle_facturas usa id_detalle_factura INT AUTO_INCREMENT como clave primaria. Las claves foráneas apuntan a factura y producto existentes. No se declara una restricción UNIQUE para el par (id_factura, id_producto), por lo que el esquema permite líneas repetidas del mismo producto.
- La aplicación solo ofrecerá eliminar clientes sin facturas y productos sin detalles. Las claves foráneas impiden eliminar registros referenciados; db.sql no declara ON DELETE CASCADE.
- La aplicación no permitirá modificar ni eliminar facturas guardadas ni sus detalles. Solo permitirá registrar el pago de una factura Pendiente y cancelar una factura Pagado o Pendiente, reponiendo el stock según las cantidades de su detalle una sola vez. db.sql no define un identificador comercial de factura ni reglas de reutilización: id_factura es la clave primaria autogenerada.
- Un producto con stock cero permanecerá visible y no se podrá agregar a la venta por decisión de la aplicación. La columna stock permite cero y la base de datos no define una etiqueta de interfaz.
- Una fecha_registro ingresada manualmente deberá ser una fecha válida no posterior a la fecha actual; podrá ser anterior para registrar clientes antiguos.

3.5.3 Formatos y límites de los datos

Los identificadores de campo se escribirán como se indica en el modelo. La aplicación manejará texto Unicode y el despliegue configurará MySQL con utf8mb4, porque db.sql no declara el juego de caracteres ni la intercalación. Si fecha_registro se omite o se deja vacía al crear un cliente, la aplicación omitirá la columna en la inserción

para que MySQL aplique DEFAULT CURRENT_DATE; no enviará NULL en ese caso.

Las siguientes restricciones se aplicarán a los campos indicados en cada fila.

Grupo de campos	Tipo	Restriccion
Identificadores	INT	id_cliente es PK y lo suministra la aplicación; id_factura, id_producto e id_detalle_factura son AUTO_INCREMENT. Las claves primarias son únicas y no nulas; db.sql no declara UNSIGNED ni restricciones de positividad.
Nombre y fecha de cliente	VARCHAR(100), DATE	clientes.nombre es obligatorio. fecha_registro tiene DEFAULT CURRENT_DATE, pero no se declara NOT NULL; la aplicación rechaza fechas posteriores a la actual.
Unidad de medida	VARCHAR(20)	productos.unidad_medida es obligatoria. No hay ENUM ni catálogo de unidades en db.sql.
Existencias	INT	productos.stock es obligatorio y CHECK (stock >= 0). No admite fracciones en el esquema.
Precio unitario	DECIMAL(10,2)	productos.precio_unitario y detalle_facturas.precio_unitario son obligatorios y CHECK (precio_unitario >= 0); rango representable: 0.00 a 99,999,999.99.

Grupo de campos	Tipo	Restriccion
Estado y fecha de factura	ENUM, DATETIME	estado_factura admite Pagado, Cancelado o Pendiente y tiene DEFAULT Pendiente. fecha_factura tiene DEFAULT CURRENT_TIMESTAMP; el script no define zona horaria.
Detalle de factura	INT, DECIMAL(10,2)	cantidad es obligatoria y CHECK (cantidad > 0); precio_unitario guarda el precio de la venta. id_detalle_factura es la PK autogenerada. No existe una clave compuesta ni una restricción de unicidad por producto.
Importes calculados	Expresión de consulta	subtotal y total no son columnas: se calculan con la cantidad y el precio_unitario guardado en detalle_facturas.
Campos no definidos	No existen en db.sql	El esquema no contiene documento, teléfono, correo, dirección, código de producto, descripción, número FAC, subtotal almacenado ni total almacenado.
Configuración del esquema	No especificada	db.sql no declara charset, collation, motor de almacenamiento, zona horaria, índices únicos de negocio ni datos iniciales. La base se llama la_unica.

3.5.4 Criterios de aceptación de la alineación con db.sql

- RF-02 y RF-11: el sistema permitirá clientes con nombres repetidos, porque db.sql no declara unicidad para nombre. Un cliente referenciado por una factura no podrá eliminarse; al consultar esa factura se verá el nombre vigente del cliente.
- RF-03, RF-05 y RF-08: productos aceptarán stock entero no negativo y precio DECIMAL(10,2) no negativo; cantidades fraccionarias, cero o negativas se rechazarán. Si el stock disponible es insuficiente, la aplicación en PHP rechazará la venta sin cambios parciales.
- RF-04 y RF-10: la aplicación en PHP actualizará el stock de manera atómica al vender y cancelar. Si una venta descontó tres unidades y luego el stock se ajustó a una unidad, cancelar la venta dejará cuatro; una segunda cancelación no cambiará el stock.
- RF-08, RF-10 y RF-14: un producto con stock cero no se podrá agregar por la interfaz; una factura Cancelado tendrá bloqueados la cancelación y el registro de pago, y una factura Pagado no ofrecerá registrar el pago. El servidor también rechazará solicitudes que intenten omitir estas reglas.
- RF-12: eliminar un cliente o producto sin referencias será posible tras confirmar. Si hay referencias en facturas o detalle_facturas, la clave foránea impedirá la eliminación.
- RF-07, RF-11 y RF-13: las consultas calcularán subtotales con el precio guardado en el detalle. Si el precio de un producto cambia de 4.800 a 5.000, una factura anterior conservará su total calculado con 4.800 y las ventas nuevas

usarán 5.000. Precio acepta hasta dos decimales y un máximo de 99,999,999.99.

- RF-03, RF-05 y RF-11: productos.unidad_medida es texto de hasta 20 caracteres. Tanto stock como cantidad son enteros; se rechazará cantidad cero o negativa y stock negativo. La base permite cualquier texto no nulo como unidad de medida.
- RNF-03, RNF-04 y RNF-07: el servicio atenderá en bucle local; ventas, pagos y cancelaciones usarán transacciones; los ajustes manuales se serializarán con movimientos concurrentes por producto y conservarán stock no negativo; una solicitud de modificación sin token CSRF válido será rechazada.
- RF-13: las vistas y procedimientos devolverán los datos descritos en 3.5.1.1. Los reportes de facturas por cliente y por estado cuentan registros de cualquier estado; vista_total_facturas y vista_unidades_productos excluyen las facturas Cancelado; reporte_facturas_pendientes solo devuelve estado Pendiente.
- RF-06 y RF-14: una venta por cobrar quedará Pendiente, descontará existencias y aparecerá en Facturas pendientes; al registrar su pago pasará a Pagado sin cambiar el stock, y una segunda solicitud de pago será rechazada.
- RF-01 y RF-11: una fecha_registro posterior a la fecha actual será rechazada; una fecha anterior será aceptada.
- RF-15: el manual de usuario y la página del video se mostrarán aunque MySQL no esté disponible.

- RNF-08: las pruebas automatizadas del repositorio terminarán sin fallos en GitHub Actions sobre una base creada con db.sql y sobre una base migrada con 001_precio_historico.sql.

3.5.5 Actualización de bases existentes

db.sql crea una base nueva y no es un script de actualización. Una base creada con la versión 1.4 o anterior del esquema se actualizará una sola vez, tras una copia de seguridad, con referencias/migraciones/001_precio_historico.sql. La migración agrega detalle_facturas.precio_unitario, lo rellena con el precio vigente del producto al momento de migrar, porque las ventas anteriores no conservaban su precio original, y redefine vista_total_facturas, vista_unidades_productos y reporte_detalle_facturas().

Referencias

Especificación de requisitos de software (SRS): La Unica. (2026). [Documento de requisitos del proyecto, versión 1.5].

Equipo del proyecto. (2026). Esquema de la base de datos La Unica [Archivo db.sql, consultado el 30 de octubre de 2026].

Equipo del proyecto. (2026). Repositorio del aplicativo La Unica [Código fuente, pruebas automatizadas y migraciones, consultado el 30 de octubre de 2026]. https://github.com/TheCarlosS5/la-unica